

Inference-Subordinate Simulation:

Decoupling Agent Decision Time from Playback Time
in 3D Environments

Faisal Akhtar

Independent Researcher

faisalakhtar12@gmail.com

[linkedin.com/in/faisalakhtar](https://www.linkedin.com/in/faisalakhtar)

May 31, 2026

Abstract

Intelligent agent simulation has historically been constrained by a fundamental coupling: the simulation must run at the speed of the agent’s decision-making process. When the decision-making agent is a large language model (LLM), this constraint becomes severe — inference latency of several seconds per decision makes real-time simulation infeasible, forcing a tradeoff between agent intelligence and simulation throughput.

We present **inference-subordinate simulation**, an architectural pattern that eliminates this constraint by inverting the relationship between simulation and inference. Rather than the simulation running continuously and the agent reacting in real time, the simulation engine advances *only when the agent issues a decision*. Each decision is logged with full state and timing metadata. Playback reads the decision log and reproduces identical behavior at any speed, without re-running inference.

We implement this architecture using Godot 4 as the simulation engine, a Python HTTP server as the inference layer, and Claude (claude-opus-4-5) as the vision-language agent. The agent observes the 3D environment through a bird’s eye camera, reasons about spatial relationships, and issues structured tool calls that advance world state. Across 26 empirical runs comprising 222 decisions, the agent achieved a 100% goal success rate with a mean inference time of 4.53 seconds per decision ($\sigma = 1.12\text{s}$) and a mean good move rate of 90.5%. Across 26 runs, the mean total inference time was 38.66 seconds per run. The same decision sequences replay in under one second at $10\times$ speed — a demonstration that playback time is fully decoupled from inference time and adjustable as a free parameter. The representative run used to illustrate this comparison was selected to match the dataset mean, not to maximize the speedup figure.

The architecture is model-agnostic at the inference layer: any vision-language model can be substituted without modifying the simulation engine or replay system. Navigation quality is model-dependent; we document the failure of a 7B local model and the successful substitution of a capable API model, treating this as an honest characterization of current model capabilities rather than an architectural limitation. We discuss applications in cinematic production, video game NPC pre-computation, reinforcement learning training data generation, and social science modeling where the real-time constraint is absent and the decoupled architecture offers substantial practical advantages.

1 Introduction

The deployment of large language models as autonomous agents in simulated environments presents a fundamental architectural tension. Simulation engines are designed around the assumption of continuous time — the world advances frame by frame, agents perceive and react within tight latency budgets, and the coherence of the simulation depends on all components operating within the same temporal envelope. Large language models violate this assumption

by design. A single inference call to a capable vision-language model requires several seconds of compute time. At this latency, real-time agent simulation is not merely difficult — it is architecturally incoherent. The simulation cannot wait several seconds per agent per decision cycle without either freezing world state, degrading to scripted fallback behavior, or abandoning the pretense of real-time operation entirely.

The standard responses to this tension are unsatisfying. Smaller, faster models reduce latency but sacrifice the reasoning capability that makes LLM agents interesting. Batching decisions introduces temporal artifacts and breaks the perception-action loop. Running inference on dedicated hardware shifts the cost without eliminating it. Pre-scripting agent behavior defeats the purpose of using a language model. None of these approaches address the root cause: the assumption that simulation time and inference time must be coupled.

1.1 The Core Insight

This paper proposes and validates a different approach. Rather than forcing the LLM to operate within the simulation’s temporal envelope, we invert the relationship: *the simulation operates within the LLM’s temporal envelope*. The engine does not advance on a clock. It advances on a decision. World state is frozen between decisions. When the LLM issues a tool call, the engine executes it, captures the resulting state, and waits for the next decision. Time inside the simulation is measured in decisions, not seconds.

This inversion has an immediate and important consequence. Because each decision is a discrete, logged event — not a continuous stream — the simulation becomes a *recording*. The decision log contains complete information: what the agent observed, what it decided, what happened, and how long inference took. Playback reads the log and reproduces the identical behavioral sequence at any speed, without re-running the LLM. The playback speed is a free parameter with no lower or upper bound. The same recorded run that took 38 seconds of inference can replay in under one second, or over ten minutes, depending on what the viewer requires.

We call this pattern **inference-subordinate simulation**. The simulation is subordinate to the inference process rather than the reverse.

1.2 Contributions

This paper makes the following contributions:

- We define and formalize the inference-subordinate simulation pattern, distinguishing it from real-time agent architectures and conventional record-replay systems.
- We present a complete implementation using Godot 4, a Python HTTP inference server, and a vision-language model (Claude claude-opus-4-5) as the agent, demonstrating the architecture on a 3D navigation task.
- We provide empirical validation across 26 runs comprising 222 agent decisions, establishing baseline metrics for inference timing, navigation quality, and the decoupling property.
- We document the model selection process honestly, including the failure of a 7B local model on spatial reasoning tasks, as evidence that navigation quality is model-dependent while the architectural pattern is model-agnostic.
- We identify and discuss a set of application domains — cinematic production, video game NPC pre-computation, reinforcement learning data generation, and social science modeling — where the absence of a real-time constraint makes the inference-subordinate pattern directly applicable.

1.3 Paper Structure

The remainder of this paper is organized as follows. Section 2 reviews related work in LLM agent architectures, game engine AI systems, and record-replay mechanisms. Section 3 describes the inference-subordinate architecture in detail. Section 4 covers the implementation. Section 5 presents empirical evaluation. Section 6 discusses applications and broader implications. Section 7 concludes.

2 Related Work

2.1 LLM Agents in Simulated Environments

The use of large language models as autonomous agents has grown rapidly since the availability of capable instruction-following models. Early work demonstrated that LLMs could decompose high-level goals into executable sub-tasks [Wei et al., 2022, Yao et al., 2023]. Subsequent systems extended this to embodied settings, placing LLM agents inside simulated environments where they must perceive, reason, and act [Park et al., 2023, Wang et al., 2023]. Generative Agents [Park et al., 2023] demonstrated that LLM-driven agents could produce believable social behavior in a simulated town, but the system required significant engineering to manage the latency of repeated inference calls, including aggressive caching and behavioral simplification. Voyager [Wang et al., 2023] deployed an LLM agent in Minecraft, again requiring careful management of the boundary between inference time and game time. In both cases, the real-time constraint is treated as a fixed engineering problem to be managed rather than an architectural assumption to be questioned.

More recent work has scaled LLM agent simulation to thousands of agents and complex social dynamics. Piao et al. [2025] simulate over 10,000 LLM-driven agents producing 5 million interactions, while Gürçan and Canbaz [2024] systematically survey architectures for integrating LLMs into agent-based social simulations. Gao et al. [2024] provide a comprehensive survey of LLM-empowered agent-based modeling across social science, epidemiology, and economics. A common challenge identified across this body of work is computational cost: LLM inference at scale strains real-time budgets, and most systems resort to batching, caching, or behavioral simplification to remain tractable. The inference-subordinate pattern we propose removes this constraint for the class of applications that do not require real-time operation.

Vision-language models extend LLM agency to visual observation, enabling agents to reason about what they see rather than relying on structured text representations of world state [OpenAI, 2023, Liu et al., 2023]. This capability is central to our architecture: the agent observes the simulation through a camera, not through an API that returns structured data. The visual grounding requirement introduces its own challenges, as we document in our model selection process (Section 4.5).

Spatial reasoning in particular remains an open challenge for vision-language models. Chen et al. [2024] demonstrate that standard VLMs perform poorly on metric spatial tasks and require specific training to develop reliable spatial judgment. Our empirical experience with a 7B local model confirms this finding: absolute left-right spatial reasoning from a bird’s eye camera was not reliably achievable at that model scale, motivating our use of a larger API-hosted model for the POC demonstration.

2.2 Game Engine AI Architectures

Traditional game AI operates within strict real-time budgets. Behavior trees [Colledanchise and Ögren, 2018], finite state machines, and utility-based systems are designed to produce decisions within milliseconds, sharing CPU time with rendering, physics, and audio. The architecture

assumes that agent intelligence and simulation throughput compete for the same computational resource within the same time slice.

Recent work has explored integrating language models into game engines as NPC controllers [Shanahan et al., 2023, Côté et al., 2019], and more recent systems have investigated generating structured game behaviors from natural language at runtime [Gallotta et al., 2025]. These approaches either simplify the environment to text-based interaction or accept degraded real-time performance. The inference-subordinate pattern we propose is orthogonal to these efforts: rather than fitting LLM inference into the real-time budget, we remove the real-time budget as a constraint entirely for the class of applications that do not require it.

2.3 Record and Replay Systems

Record and replay mechanisms are well established in software engineering [Adl-Tabatabai and Gross, 1996], game development, and distributed systems [Qin et al., 2005] for purposes of debugging, testing, and deterministic reproduction of behavior. Deterministic replay in games typically records player inputs and re-simulates physics to reproduce game state. Our approach differs in two important ways. First, the unit of recording is an agent decision with associated inference metadata, not a raw input stream. Second, replay does not re-simulate — it reads behavioral outcomes directly from the log, making replay speed fully independent of simulation complexity.

2.4 Discrete-Event Simulation

Discrete-event simulation (DES) is a well-established computational paradigm in which the system state advances only when events occur, rather than progressing on a continuous clock [Banks et al., 1996]. In this sense, DES systems are structurally event-subordinate by design — a property superficially similar to the inference-subordinate pattern proposed here. However, the analogy breaks down on three critical dimensions. First, in DES the events themselves are computationally trivial — they are typically state transitions defined by simple rules or probability distributions, not the outputs of multi-second inference calls to large neural models. The subordination in DES is a scheduling convenience; the subordination in inference-subordinate simulation is a computational necessity imposed by the inference latency of capable vision-language models. Second, DES systems do not produce observation-grounded decisions: agents in DES do not perceive a rendered scene, reason about what they observe, and issue structured tool calls. The perception-action loop — camera, encoding, LLM reasoning, tool call extraction — is absent from DES entirely. Third, and most relevant to this paper, DES systems do not produce a decision log that records inference timing metadata alongside behavioral outcomes. The log in our architecture is not merely a record of what happened; it records how long each decision took to produce, enabling the empirical comparison between inference time and replay time that constitutes the paper’s central result. A DES event log contains no such timing information because DES events have no inference cost to measure.

2.5 Positioning This Work

The inference-subordinate pattern sits at the intersection of these three bodies of work but is not reducible to any of them. It is not a real-time agent architecture — it explicitly abandons the real-time constraint. It is not a conventional record-replay system — the recording is a first-class output rather than a debugging artifact. It is not a standard LLM agent framework — the simulation engine is a passive recorder rather than an active environment that the agent must keep pace with. The contribution is the inversion of the temporal relationship between simulation and inference, and the set of applications this inversion makes tractable.

3 Architecture

3.1 System Overview

The inference-subordinate simulation architecture consists of three components: a **simulation engine**, an **inference server**, and a **decision log**. Figure 1 illustrates the system. The simulation engine maintains world state and renders observations. The inference server hosts the vision-language model and processes observations into tool calls. The decision log is a persistent, append-only record of every agent decision with associated state and timing metadata.

The defining property of the architecture is the direction of control. The simulation engine does not drive inference — it waits for it. The engine captures an observation frame, transmits it to the inference server, and suspends world state advancement until a tool call is returned. Simulation time is therefore measured in decisions, not in wall clock seconds. A run that requires 10 decisions and 45 seconds of inference time contains exactly 10 logged state transitions, regardless of how long each inference call took.

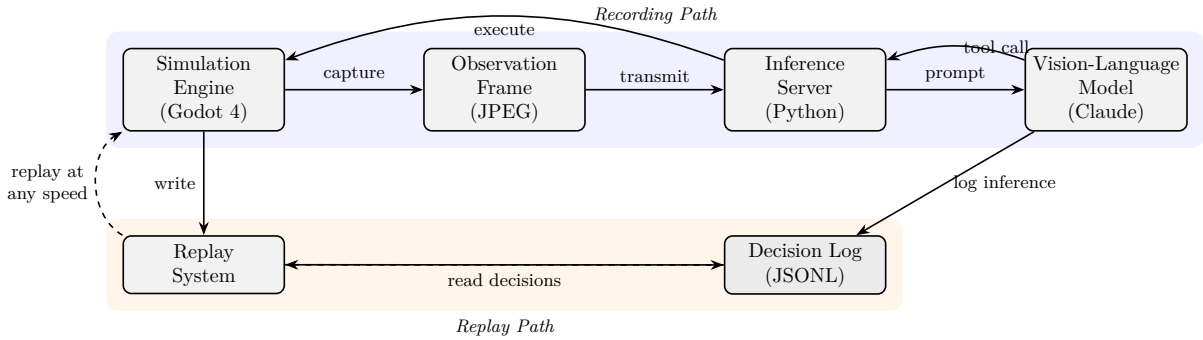


Figure 1: The inference-subordinate simulation architecture. *Recording path* (blue): the engine captures an observation, the inference server queries the vision-language model, the returned tool call is executed and logged with timing metadata. World state is frozen during inference. *Replay path* (orange): the replay system reads the decision log and reproduces the behavioral sequence at any speed, bypassing the inference server entirely.

3.2 The Inference-Subordinate Engine Loop

The core loop differs fundamentally from a real-time agent loop. In a conventional real-time architecture, the engine runs continuously and the agent must produce decisions within a fixed time budget. In the inference-subordinate architecture, the engine is event-driven: it advances exactly one step per decision event. The loop is as follows:

1. The engine captures an observation from the active camera viewport and transmits it to the inference server.
2. The inference server encodes the observation, constructs a prompt incorporating world description and decision history, and submits the request to the vision-language model.
3. The model reasons about the observation and returns a structured tool call specifying the action to execute.
4. The engine executes the tool call, advances world state, and writes a decision entry to the log.
5. The engine captures a new observation and the loop repeats.

World state is frozen between steps 1 and 4. No simulation advancement occurs during inference. The engine does not time out, degrade, or fall back to scripted behavior while waiting. It simply waits. This waiting is not a limitation — it is the mechanism by which inference time is decoupled from simulation time.

3.3 The Observation Layer

At each decision cycle, the engine captures the current viewport as a JPEG-encoded image and transmits it to the inference server over a local HTTP connection. Two camera configurations exist in the scene: a bird’s eye camera positioned directly above the environment providing a top-down orthographic-equivalent view for the agent, and a ground-level camera providing a perspective view for human review and replay. The agent receives only the bird’s eye view. The ground-level camera is activated during replay to provide a visually informative presentation of the recorded behavior.

The inference server maintains a rolling buffer of the previous frame, enabling the model to receive both the current and previous observation at each decision step. This two-frame context supports temporal reasoning about movement and change between decisions.

3.4 The Tool Call Interface

The agent communicates with the simulation engine exclusively through structured tool calls. In the current implementation, a single primitive is defined:

```
{"tool": "move", "params": {"dx": float, "dz": float}}
```

The `dx` and `dz` parameters specify displacement in the X and Z axes of the world coordinate system respectively. The engine clamps movements to the boundary of the navigable plane. The tool call interface is intentionally minimal: the contribution of this paper is the architectural pattern, not the action space. Richer action spaces, hierarchical tool sets, and multi-agent tool coordination are straightforward extensions.

The inference server extracts tool calls from model output using a four-pass parser that handles JSON in multiple formats, including prefixed output (`MOVE: {...}`), bare JSON objects, markdown-fenced blocks, and a reconstruction fallback that extracts `dx/dz` values from free-form text. This robustness layer ensures that variance in model output formatting does not interrupt the decision loop.

3.5 The Decision Log

Every agent decision is written to an append-only JSONL file. Each entry records the complete information required to reproduce and analyze the decision:

```
{
  "type": "decision",
  "action": "move",
  "dx": -0.5,
  "dz": -0.5,
  "inference_time": 4.63,
  "position": {"x": ..., "y": ..., "z": ...},
  "distance_to_goal": 2.30,
  "goal": {"x": ..., "y": ..., "z": ...},
  "frame": 93,
  "timestamp": 1780088929.478
}
```

The log is prefixed by a `run_header` entry recording the random seed, initial puppet position, and goal position, enabling exact reproduction of the spatial configuration. It is suffixed by

a `run_summary` entry recording total decision count, aggregate inference time, and wall clock recording duration. The log is the complete artifact of a recording run — the replay system requires only this file and the simulation scene to reproduce the full behavioral sequence.

3.6 The Replay System

The replay system reads the decision log and reproduces the recorded behavioral sequence without invoking the inference server. On initialization, the replay system reads the `run_header` to restore the exact starting positions of all scene objects. It then steps through the `decision` entries in sequence, applying each recorded dx/dz displacement to the agent at a configurable interval.

The replay speed is controlled by a single parameter: the interval between decision steps. This parameter has no lower bound other than frame rate and no upper bound other than viewer preference. The same recorded run can replay at human-perceptible speed for demonstration purposes, at maximum frame rate for bulk analysis, or at $0.1\times$ speed for detailed inspection of individual decisions. Crucially, this parameter is entirely independent of the inference time recorded in the log. The recorded inference latencies are metadata — they document what happened during recording but place no constraint on playback.

At replay completion, the system reports the replay duration alongside the recorded inference time and recording wall clock duration, providing a direct empirical comparison of the three time measurements that constitute the paper’s core claim.

4 Implementation

4.1 Stack Overview

The implementation uses three components: Godot 4 as the simulation engine, a Python HTTP server as the inference layer, and the Claude API (`claude-opus-4-5`) as the vision-language model. Communication between the engine and inference server uses a lightweight HTTP polling protocol over localhost. No external networking, message queues, or orchestration frameworks are required. The full system runs on a single consumer workstation.

4.2 Scene Configuration

The simulation scene contains three elements: a flat ground plane defining the navigable area, a white capsule mesh representing the agent (referred to as the puppet), and a red sphere representing the goal. The navigable plane spans $[-5.0, 5.0]$ on both the X and Z axes. At the start of each run, the puppet and goal are placed at random positions drawn from $\mathcal{U}(-3.0, 3.0)$ on both axes, subject to a minimum separation constraint of 4.0 units. This constraint ensures that each run requires multiple decisions to complete, producing a non-trivial behavioral sequence for logging and replay.

Agent movement is parameterized by a step size of 0.5 units per decision in each axis. This value was selected empirically to produce convergent navigation in 6–15 decisions from typical starting distances of 4.0–6.0 units. Larger step sizes produced oscillation as the agent overshot the goal; smaller step sizes increased decision count without improving trajectory quality.

4.3 Camera Setup

Two cameras exist in the scene simultaneously. The **BirdsEyeCamera** is positioned at $(0, 20, 0)$ with a -90° rotation on the X axis, providing a top-down view of the entire navigable plane. This camera is the active viewport during recording and is the source of all observation frames transmitted to the inference server. The **GroundCamera** is positioned at $(0, 4, 8)$ with a -20°

rotation on the X axis, providing a perspective view of the scene at ground level. This camera is inactive during recording. It is activated during replay to provide a visually informative presentation of the recorded behavior, demonstrating that the camera configuration is a free parameter of the replay system independent of the recording configuration.

This separation has a precise motivation. The bird’s eye view decomposes the 3D navigation task into a 2D spatial reasoning problem: the agent need only determine whether the goal is left, right, above, or below its current position in the image plane. Ground-level perspective introduces depth ambiguity and occlusion that significantly increases the difficulty of the visual reasoning task. The camera is therefore an active architectural choice, not a default.

4.4 Communication Protocol

The inference server exposes three HTTP endpoints. The `/frame` endpoint accepts POST requests containing JPEG-encoded viewport captures. The `/poll` endpoint accepts GET requests from the engine; it returns a tool call envelope when inference is complete, or HTTP 204 when no decision is ready. The `/result` endpoint accepts POST requests containing the outcome of each executed tool call, including the updated distance to goal. This three-endpoint design cleanly separates frame delivery, decision retrieval, and result reporting.

The engine polls `/poll` at 0.5 second intervals after frame delivery. The inference server sets a `new_frame_received` flag on frame receipt and clears it before beginning inference, preventing stale frame processing. The complete round trip — frame capture, transmission, inference, tool call execution, and result reporting — constitutes one decision cycle.

4.5 Model Selection

Model selection was a non-trivial decision that produced empirical findings relevant to the architecture’s deployment requirements. The initial implementation used LLaVA 7B (v1.6-mistral, Q4_K_M quantization) running locally via Ollama on an NVIDIA GTX 1060 6GB. A 20-run evaluation on a fixed spatial configuration produced 11 LEFT and 9 RIGHT responses — statistically indistinguishable from random. This failure is consistent with published findings on spatial reasoning limitations in sub-10B vision-language models [Chen et al., 2024]: absolute directional judgment from overhead imagery requires spatial reasoning capability that 7B models do not reliably possess.

Qwen2.5-VL 7B and Llama 3.2 Vision 11B were evaluated as alternatives. Both exceeded the 6GB VRAM budget of the available hardware and fell back to CPU inference, producing latencies incompatible with practical data collection. These constraints are hardware-specific and not architectural: larger local models would be viable on hardware with sufficient VRAM.

The final implementation uses Claude (`claude-opus-4-5`) via API. This substitution required changing a single component — the inference call in `mcp_client.py` — with no modifications to the simulation engine, communication protocol, decision log format, or replay system. This substitution demonstrates the model-agnosticism of the architecture directly: the engine has no dependency on which model occupies the inference layer.

We document this selection process honestly rather than presenting only the successful configuration. The architectural claim — that inference time and playback time are decoupled — holds regardless of which model is used. The navigation quality claim — that the agent successfully reaches the goal — is model-dependent. Deployers of this architecture must select a model with sufficient visual spatial reasoning capability for their specific task.

4.6 Prompt Engineering

The prompt presented to the model at each decision cycle consists of four layers. Layer 1 provides a world description: the agent is told it is observing a top-down bird’s eye view, identifies the

white capsule as the controllable agent and the red dot as the static goal, and specifies the coordinate system mapping image directions to dx/dz values. Layer 2 provides decision history: the last five moves are listed with their outcomes expressed as distance changes, and heuristic warnings are generated for oscillation and boundary collision patterns. Layer 3 presents a structured four-step observation template requiring the model to explicitly locate both objects, determine their relative positions, derive movement values, and output a formatted tool call. Layer 4 enforces output format: the **MOVE:** prefix line is mandatory and values must match the derivation in Step 3.

This structured elicitation approach — requiring the model to reason step by step before committing to a tool call — produced substantially more reliable spatial judgments than direct instruction. The four-pass JSON extraction pipeline handles variance in model output format without interrupting the decision loop.

5 Evaluation

5.1 Experimental Setup

All experiments were conducted on a single consumer workstation running Windows 11 with an NVIDIA GTX 1060 6GB GPU. The simulation engine ran in Godot 4.6.2 in windowed mode at 1152×648 resolution. Viewport captures were resized to 336×336 pixels before transmission to the inference server. The inference server ran as a local Python process communicating with the Claude API (`claude-opus-4-5`) over HTTPS.

Each run was initialized with a fresh random seed derived from the system clock. Puppet and goal positions were drawn from $\mathcal{U}(-3.0, 3.0)$ on both axes subject to a minimum separation of 4.0 units. A run was considered complete when the agent reached within 0.8 units of the goal center. No maximum decision count was imposed — runs continued until the goal was reached. All 26 runs reached the goal.

5.2 Dataset

The dataset comprises 26 complete runs and 222 individual agent decisions. Table 1 summarizes the run-level statistics. The full dataset is available alongside the implementation at the project repository.

Table 1: Run-level dataset summary (26 runs, 222 decisions).

Metric	Mean	Std Dev	Range
Decisions per run	8.54	2.64	5 – 17
Starting distance (units)	4.59	0.56	4.01 – 6.10
Total inference time (s)	38.66	14.38	23.26 – 92.47
Recording duration (s)	72.71	26.08	42.32 – 143.44
Good move rate	0.921	0.099	0.706 – 1.000
Path efficiency	1.065	0.160	0.550 – 1.212

5.3 Inference Timing

Across all 222 decisions, the mean inference time per decision was 4.53 seconds ($\sigma = 1.12s$, median = 4.25s, range 2.72s – 12.08s). Figure 2 shows the distribution of per-decision inference times. The distribution is right-skewed, with the majority of decisions completing in the 3.5s – 5.5s range and a small number of outliers exceeding 7 seconds.

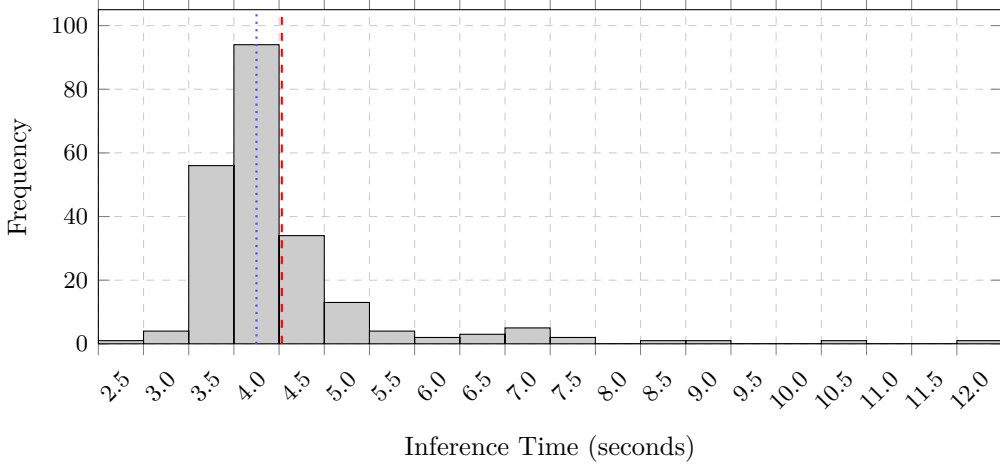


Figure 2: Distribution of per-decision inference times across all 222 decisions (26 runs). The distribution is concentrated in the 3.5s–5.0s range (184/222 decisions, 82.9%) with a right tail reflecting occasional network latency spikes. Red dashed line: mean (4.53s). Blue dotted line: median (4.25s). Outliers above 7s account for 10 decisions (4.5%) and correspond to API latency variance rather than model compute differences.

The inference-subordinate architecture accommodates this variance transparently. When Run 26 produced a single inference call requiring 12.08 seconds — more than twice the mean — the simulation engine waited without degradation. No frame was dropped, no state was corrupted, and no fallback behavior was triggered. The decision was logged identically to all others. This behavior is not a robustness feature that required engineering; it is a direct consequence of the architectural pattern. An engine that advances only on decisions cannot be disrupted by decision latency.

5.4 Navigation Quality

Across 222 decisions, 201 were good moves (decreased distance to goal by more than 0.05 units) and 21 were bad moves (increased distance by more than 0.05 units), yielding an overall good move rate of 90.5%. The mean per-run good move rate was 92.1% ($\sigma = 9.9\%$). Fourteen of 26 runs (53.8%) completed with a perfect good move rate of 100%. All 26 runs reached the goal — a 100% success rate across all experimental configurations.

Mean path efficiency was 1.065 ($\sigma = 0.160$), where a value of 1.0 indicates a straight-line trajectory and values above 1.0 indicate minor deviation. The near-unity mean path efficiency confirms that the agent is navigating directly toward the goal rather than wandering, and that the good move rate reflects genuine spatial reasoning rather than lucky convergence.

These results establish that the visual perception-action loop is functioning correctly: the agent observes the scene, determines the relative position of the goal, and issues appropriate movement commands. This is a necessary condition for the paper’s architectural claim — if the agent were navigating randomly, the decision log would not constitute a meaningful behavioral record.

5.5 The Oscillation Case

Run 24 produced 17 decisions with a good move rate of 70.6% — the lowest in the dataset — as the agent entered an oscillation pattern near the goal, alternating between two positions at distances 1.06 and 1.60 from the goal for seven consecutive decisions before finally converging. We report this run without modification. It illustrates two properties of the architecture simultaneously. First, the decision log faithfully records imperfect navigation: the oscillation is

visible in the log and fully reproducible in replay. Second, the architecture places no constraint on navigation quality: a less capable model, a harder task, or an adversarial configuration would produce lower quality navigation, and the recording and replay system would handle it identically. The architecture is indifferent to the quality of the decisions it records.

5.6 Replay Demonstration

The mean total inference time across all 26 runs was 38.66 seconds ($\sigma = 14.38s$). The representative run selected for Table 2 matches this mean exactly, and is not an outlier chosen to maximize the speedup figure. This run of 8 decisions replays in under one second at maximum speed — a speedup exceeding $38\times$ relative to inference time. Runs with higher decision counts and longer inference times produce proportionally larger speedups; Run 24, with 92.47 seconds of total inference time, replays in under two seconds at maximum speed regardless of its inference duration. The replay speed is adjustable in real time via a continuous parameter ranging from $0.1\times$ to $10\times$ relative to a one-second-per-decision baseline. At $0.1\times$ speed, the same run takes over 80 seconds, allowing frame-by-frame inspection of individual decisions. At $10\times$ speed, it completes in under one second. The recorded inference times are unaffected by this parameter — they are metadata in the log, not constraints on playback.

Table 2 presents the three time measurements that constitute the paper’s core empirical result.

Table 2: Timing comparison for a representative 8-decision run demonstrating the decoupling property.

Measurement	Value
Total inference time (sum of 8 LLM calls)	38.66s
Recording wall clock duration	61.44s
Replay duration at $10\times$ speed	$<1s$
Replay duration at $1\times$ speed	$\sim 8s$
Replay duration at $0.1\times$ speed	$\sim 80s$
Inference-to-replay speedup ($10\times$)	$>38\times$

The key observation is that the three rightmost rows of Table 2 are free parameters. They can be set to any value the application requires. The top two rows are fixed properties of the recording. The decoupling is total: no property of the playback depends on any property of the inference.

5.7 Reproducibility

The complete implementation is available at <https://github.com/fakhtar/tilismpoc1> including the Godot 4 scene files, Python inference server, replay system, and the full dataset of 26 runs in CSV format. Each run in the dataset includes the random seed used to initialize puppet and goal positions, enabling exact reproduction of the spatial configuration of any recorded run. The decision log format is documented in Section 3.5.

6 Applications and Implications

The inference-subordinate architecture removes the real-time constraint from agent simulation. This removal is consequential only for applications where the real-time constraint is currently a binding limitation. We identify four such domains where the architectural pattern is directly

applicable and where the absence of a real-time requirement is either inherent to the domain or already accepted practice.

6.1 Cinematic and Film Production

Film production has no real-time constraint by definition. A visual effects shot can take hours or days to render; the behavioral intelligence layer should have the same freedom. Current practice for crowd simulation, battle sequences, and complex choreographed scenes relies on rule-based behavioral systems and hand-authored scripts precisely because LLM inference latency is incompatible with interactive simulation tools. The inference-subordinate architecture removes this incompatibility. A director describes a scenario; LLM-driven agents simulate it at inference speed; the resulting behavioral record is rendered to footage at any frame rate the pipeline requires. The intelligence of the simulation is bounded only by the capability of the model, not by the render budget or the frame rate.

The decision log has an additional value in production contexts: it is an auditable record of every agent decision. A sequence can be replayed from any agent’s perspective, inspected at any speed, and modified by editing the log rather than re-running inference. This auditability is a natural consequence of the architecture rather than an additional engineering investment.

6.2 Video Game NPC Pre-Computation

The video game application of the inference-subordinate architecture deserves precise framing because the mechanism is non-obvious. We call it **deferred behavioral simulation**. The game engine operates in two modes: a slow inference-subordinate simulation mode that runs as a background process during loading screens, idle time, or off-peak server hours, and a fast playback mode that the player experiences in real time. The handoff between modes is the decision log.

This mechanism produces NPC behavior that is experientially intelligent but computationally pre-paid. A player encounters an NPC whose daily routine, faction decisions, and responses to world events were simulated hours earlier by an LLM reasoning about the game state. The player experiences the playback. The inference already happened. The real-time budget is spent only on playback, which is trivially cheap.

This approach has a specific advantage beyond intelligence quality: NPC behavior becomes deterministic and auditable. Every decision is logged. Game designers can replay exactly what happened and why, inspect individual decisions, and identify behavioral anomalies without re-running inference. For open world games where NPC daily routines, faction dynamics, and emergent world events are central to the player experience, this architecture offers a path to genuine behavioral intelligence that scripted behavior trees and finite state machines cannot provide.

6.3 Reinforcement Learning Training Data Generation

A persistent bottleneck in reinforcement learning is the cost of generating diverse, high-quality behavioral training data. Collecting human demonstration data is expensive and does not scale. Scripted agents produce behavioral data that is systematically limited by the quality of the scripts. LLM-driven agents conditioned on task descriptions, domain knowledge, and behavioral objectives can produce demonstrations that are qualitatively richer than scripted alternatives, but only if the inference cost can be managed.

The inference-subordinate architecture makes LLM-generated demonstration data tractable. A simulation runs at inference speed, accumulates a decision log of intelligent agent behavior, and the log becomes a labeled training trajectory. The log format records the observation, the decision, and the outcome at every step — precisely the (s, a, s') tuples that RL algorithms

require. Generation can run overnight on modest hardware and produce arbitrarily large libraries of diverse behavioral demonstrations. This is particularly relevant for robotics simulation, where the sim-to-real gap is partly a behavioral diversity problem: policies trained on scripted demonstrations fail to generalize because the demonstrations do not cover the space of intelligent responses to unexpected situations [Gao et al., 2024].

6.4 Social Science and Epidemiological Modeling

Agent-based models are an established methodology in computational social science, epidemiology, economics, and urban planning [Gürçan and Canbaz, 2024, Piao et al., 2025]. Current agent-based models use rule-based agents whose behavioral realism is limited by the expressiveness of hand-authored rules. Replacing rule-based agents with LLM agents conditioned on domain literature, demographic profiles, and behavioral theory produces more realistic emergent dynamics — but at an inference cost that makes large-scale simulation on research hardware appear intractable.

The inference-subordinate architecture makes this computationally feasible. A social simulation with hundreds or thousands of LLM-driven agents runs at inference speed — potentially over hours or days — and produces a complete behavioral record. Researchers analyze and replay the recording rather than the live simulation. The temporal scale of the simulation is decoupled from the temporal scale of the analysis. A city evacuation model, an epidemic spread simulation, or an economic shock propagation study can run as slowly as the inference budget requires and be replayed and analyzed at any speed thereafter.

Recent work has demonstrated the potential of LLM agents in social simulation contexts [Piao et al., 2025, Gürçan and Canbaz, 2024, Gao et al., 2024], but has not addressed the inference cost problem architecturally. The inference-subordinate pattern is a direct solution to the computational bottleneck these works identify.

6.5 The General Principle

These four applications share a common structure. Each involves a domain where: (1) real-time operation is not required; (2) behavioral intelligence is currently limited by the cost of achieving it in real time; and (3) a recorded behavioral artifact has value independent of the process that produced it. The inference-subordinate architecture is applicable wherever this structure holds. The four domains described here are illustrative, not exhaustive.

7 Discussion

7.1 What the Results Establish

The empirical results reported in Section 5 support three claims of different character. The first is architectural: a simulation engine can be operated as an inference-subordinate recording system, advancing only on agent decisions and producing a log that replays at any speed without re-running inference. This claim is established by the working implementation. It does not require statistical support — it is either true of the system or it is not, and the replay demonstration confirms that it is.

The second claim is behavioral: a vision-language model operating through this architecture can successfully navigate a 3D environment using visual observations alone, achieving a 100% goal success rate across 26 runs with a 90.5% good move rate. This claim is statistical and the dataset supports it. The mean path efficiency of 1.065 confirms that the agent is reasoning about the scene rather than performing a random walk — a random walk would produce path efficiencies substantially below 1.0 and a good move rate near 50%.

The third claim is about the decoupling property specifically: playback time is a free parameter independent of inference time. This is demonstrated by the variable speed replay

system. The same 8-decision trajectory that required 38.66 seconds of inference replays in under one second at maximum speed and over 80 seconds at minimum speed. Neither bound is determined by the inference time. This is the paper’s central contribution stated as an empirical observation.

7.2 Model-Agnosticism and Model-Dependence

A distinction introduced in Section 4.5 warrants explicit discussion. The architecture is model-agnostic in a precise sense: the simulation engine, communication protocol, decision log format, and replay system have no dependency on which model occupies the inference layer. Substituting LLaVA 7B for Claude required changing a single function in a single file. The rest of the system was unaffected.

Navigation quality, however, is model-dependent. LLaVA 7B failed at the spatial reasoning task entirely. Claude succeeded with high consistency. This is not a contradiction — it is a characterization of the deployment requirement. The architecture imposes a capability floor on the inference layer: the model must be able to perform the perceptual reasoning required by the task. What the architecture does not impose is a latency ceiling. A model that takes 12 seconds per decision, as occurred in Run 26, is fully compatible with the architecture. A model that cannot determine left from right, as LLaVA 7B demonstrated, is not — but this is a model limitation, not an architectural one.

This distinction matters for deployers. The question to ask when selecting a model for an inference-subordinate simulation is not *how fast is it?* but *can it perform the perceptual task?* Speed is irrelevant to correctness in this architecture. Capability is not.

7.3 The Decision Log as a First-Class Artifact

Conventional record-replay systems treat recordings as debugging artifacts: they are produced when something goes wrong and discarded otherwise. The decision log in the inference-subordinate architecture is a first-class output. It is the primary product of a recording run, and its value is independent of whether the run succeeded or failed.

Run 24 illustrates this. The oscillation pattern visible in that run’s log — seven decisions alternating between two positions before convergence — is not a failure to be hidden. It is an accurate record of the agent’s reasoning under a difficult spatial configuration. A researcher analyzing the log can identify exactly where the model lost directional consistency, at what distance the oscillation began, and how it resolved. This analysis is available because the log is complete and faithful. An architecture that discarded decisions during poor performance would not provide this.

The completeness of the log also has implications for accountability. In the video game application described in Section 6.2, every NPC decision is logged and replayable. In the social science application, every agent decision in a simulation is auditable. The value of the log in these contexts derives from the same property that makes it useful for debugging: it records everything, always, regardless of outcome.

7.4 Limitations

We identify four limitations of the current work that bound the scope of the claims.

Single task and environment. All 26 runs used the same scene configuration: a flat plane, a single capsule agent, and a single goal sphere. The architectural claim generalizes beyond this configuration by construction — the engine-inference separation does not depend on scene complexity — but the behavioral claim (90.5% good move rate, 100% success rate) is specific to this task. More complex environments, multiple agents, and dynamic obstacles would require separate evaluation.

Single model. All successful runs used a single model (`claude-opus-4-5`). The model-agnosticism claim is supported by the failed LLaVA substitution and the clean API swap, but a complete characterization would evaluate multiple capable models and report variance in navigation quality across them.

Single agent. The current implementation involves one agent making one decision per cycle. Multi-agent inference-subordinate simulation — where multiple agents observe, reason, and act, potentially in parallel — introduces coordination questions that this work does not address.

Multi-agent extension. The single-agent limitation does not imply that multi-agent inference-subordinate simulation requires global simulation pausing. A multi-agent extension supports two inference modes: synchronous, where a blocking decision gates that agent’s state transitions, and asynchronous, where non-blocking decisions are submitted to the inference endpoint and applied upon return while other agents continue. The simulation clock remains inference-subordinate at the per-agent level rather than globally, preserving the decoupling property while eliminating unnecessary pauses. This extension is a natural direction for future work.

No sim-to-real evaluation. The RL training data generation application described in Section 6.3 assumes that LLM-generated behavioral demonstrations transfer usefully to policy training. This assumption is theoretically motivated but not empirically evaluated here. It is a natural direction for follow-on work.

7.5 Relation to Paper Two

This paper establishes the decoupling property in two dimensions: inference time and playback time. A natural extension addresses a third dimension: the observation space. The current implementation uses a single bird’s eye camera, which decomposes 3D navigation into a 2D spatial reasoning problem by design. Paper Two of this series will examine orthogonal multi-view observation: the hypothesis that providing a vision-language model with simultaneous bird’s eye and ground-level views decomposes 3D spatial reasoning into independent 2D problems, one per view, reducing the reasoning burden on the model and improving navigation quality in environments where a single view is insufficient. The inference-subordinate architecture established here is the foundation on which that work builds.

8 Conclusion

Intelligent agent simulation has been constrained by an assumption so deeply embedded in simulation architecture that it is rarely stated explicitly: that the simulation clock and the agent decision clock must run together. This assumption is natural when agents are rule-based systems that decide in microseconds. It becomes a binding limitation when agents are large language models that decide in seconds. The result has been a forced tradeoff between agent intelligence and simulation throughput that has no principled justification — only an architectural one.

This paper has proposed, implemented, and empirically validated a pattern that dissolves this tradeoff by inverting the relationship between simulation and inference. In an inference-subordinate simulation, the engine does not drive the agent — the agent drives the engine. World state advances exactly once per decision. Inference latency, whether two seconds or twelve, is simply the time between advances. The simulation is indifferent to it.

The consequence of this inversion is that simulation time becomes a recording. Every decision is logged with complete state and timing metadata. Replay reads the log and reproduces the identical behavioral sequence at any speed the application requires, without re-running the model that produced it. The playback speed is a free parameter with no connection to the inference time recorded in the log.

Across 26 empirical runs comprising 222 agent decisions, the implementation demonstrated a 100% goal success rate with a mean good move rate of 90.5% and a mean path efficiency of 1.065. Mean inference time per decision was 4.53 seconds ($\sigma = 1.12\text{s}$). The same behavioral sequences that required a mean of 38.66 seconds of inference to produce replay in under one second at maximum speed — a speedup exceeding $38\times$. The architecture accommodated inference latency variance of 2.72s to 12.08s without degradation, fallback, or modification.

The model selection process documented here establishes an important constraint for practitioners: the architecture is model-agnostic, but navigation quality is model-dependent. A 7B local model failed the spatial reasoning task entirely. A capable API-hosted model succeeded with high consistency. The relevant question when deploying this architecture is not how fast the model is, but whether it can perform the perceptual task. Speed is irrelevant. Capability is not.

The applications identified in Section 6 — cinematic production, video game NPC pre-computation, reinforcement learning data generation, and social science modeling — share a common structure: real-time operation is not required, behavioral intelligence is currently limited by the cost of achieving it in real time, and a recorded behavioral artifact has value independent of how it was produced. The inference-subordinate pattern is directly applicable wherever this structure holds.

The work presented here is a proof of concept. The scene is simple, the task is constrained, and the agent is a single entity navigating a flat plane. These limitations are intentional: the contribution is the architectural pattern, not a benchmark result, and a minimal implementation is the clearest demonstration of a minimal claim. The claim is narrow and the evidence for it is complete: inference time and playback time can be fully decoupled, and decoupling them does not require a more complex system than the one described here. It requires only the decision to stop treating the simulation clock and the agent decision clock as the same thing.

Paper Two of this series will extend the observation model, examining whether orthogonal multi-view cameras decompose 3D spatial reasoning into independent 2D problems and improve navigation quality in environments where a single overhead view is insufficient. The inference-subordinate architecture established here is the substrate on which that investigation rests.

References

- A.-R. Adl-Tabatabai and T. Gross. Debugging optimized code with dynamic deoptimization. In *ACM SIGPLAN Notices*, 1996. Record-replay systems lineage.
- J. Banks, J. S. Carson, B. L. Nelson, and D. M. Nicol. *Discrete-Event System Simulation*. Prentice Hall, Upper Saddle River, NJ, 2nd edition, 1996.
- B. Chen, Z. Xu, S. Kirmani, B. Ichter, D. Driess, P. Florence, D. Sadigh, L. Guibas, and F. Xia. SpatialVLM: Endowing vision-language models with spatial reasoning capabilities. *arXiv preprint arXiv:2406.13537*, 2024.
- M. Colledanchise and P. Ögren. *Behavior trees in robotics and AI: An introduction*. CRC Press, 2018.
- M.-A. Côté, Á. Kádár, X. Yuan, B. Kybartas, T. Barnes, E. Fine, J. Moore, M. Hausknecht, L. El Asri, M. Adada, et al. TextWorld: A learning environment for text-based games. In *Computer Games Workshop at ICML/IJCAI*, 2019.
- R. Gallotta et al. Real-time world crafting: Generating structured game behaviors from natural language with large language models. *arXiv preprint arXiv:2510.16952*, 2025.
- C. Gao, X. Lan, N. Li, Y. Yuan, J. Ding, Z. Zhou, F. Xu, and Y. Li. Large language models empowered agent-based modeling and simulation: a survey and perspectives. *Humanities and Social Sciences Communications*, 11, 2024. doi: 10.1057/s41599-024-03611-3.

- Ö. Gürcan and M. A. Canbaz. LLM-augmented agent-based modelling for social simulations: Challenges and opportunities. *arXiv preprint arXiv:2405.06700*, 2024.
- H. Liu, C. Li, Q. Wu, and Y. J. Lee. Visual instruction tuning. *Advances in Neural Information Processing Systems*, 36, 2023.
- OpenAI. GPT-4 technical report. Technical report, OpenAI, 2023. arXiv preprint arXiv:2303.08774.
- J. S. Park, J. C. O’Brien, C. J. Cai, M. R. Morris, P. Liang, and M. S. Bernstein. Generative agents: Interactive simulacra of human behavior. In *Proceedings of the 36th Annual ACM Symposium on User Interface Software and Technology*, pages 1–22, 2023.
- J. Piao et al. AgentSociety: Large-scale simulation of LLM-driven generative agents advances understanding of human behaviors and society. *arXiv preprint arXiv:2502.08691*, 2025.
- F. Qin, S. Joseph, Y. Pan, and Y. Zhou. Rx: Treating bugs as allergies — a safe method to survive software failures. In *ACM SIGOPS Operating Systems Review*, volume 39, pages 235–248, 2005.
- M. Shanahan, K. McDonell, and L. Reynolds. Role play with large language models. *Nature*, 623(7987):493–498, 2023.
- G. Wang, Y. Xie, Y. Jiang, A. Mandlekar, C. Xiao, Y. Zhu, L. Fan, and A. Anandkumar. Voyager: An open-ended embodied agent with large language models. *arXiv preprint arXiv:2305.16291*, 2023.
- J. Wei, X. Wang, D. Schuurmans, M. Bosma, F. Xia, E. Chi, Q. V. Le, D. Zhou, et al. Chain-of-thought prompting elicits reasoning in large language models. *Advances in Neural Information Processing Systems*, 35:24824–24837, 2022.
- S. Yao, J. Zhao, D. Yu, N. Du, I. Shafran, K. Narasimhan, and Y. Cao. ReAct: Synergizing reasoning and acting in language models. *arXiv preprint arXiv:2210.03629*, 2023.